DJI RoboMaster AI Challenge Technical Report

Hopkins AI[1]

***Abstract*— This document is a submission as the technical report for 2018 DJI RoboMaster AI Challenge. The paper reports the progress that we have made so far in both Hardware and Software development. It also briefly talks about the next step that we plans to take to prepare for the competition at Brisbane in May. The accompanying video can be found at our YouTube channel.**

I. INTRODUCTION

Artificial Intelligence is an emerging field with exciting development in the recent years. As one of the best attempts to build a standard platform for AI algorithm developing, DJI RoboMaster AI Challenge is a great opportunity for us to develop exciting algorithms addressing real world issue. In this article, we will report the progress that we have made thus far in the preparation for the RoboMaster AI Challenge. The report is divided into Hardware and Software sections, and both have subsections that further elaborate the technical details.

II. HARDWARE

This section talks about the progress and effort that we have made in terms of the hardware, to achieve the goals and overcome challenges posted by the challenge.The main progress and contributions that we made can be summarized into four parts: 1. polished printable part adapter designs 2. reported on examined part selection process 3. documented debugging/troubleshooting process 4. construction of a mock arena and performed tests.

*This work was supported by the RoboMaster Organizing Committee, and Johns Hopkins University

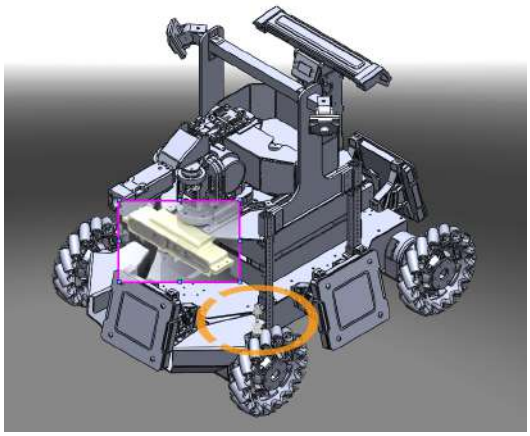
[1]Hopkins AI is a student group formed by students from Electrical and Computer Engineering, Mechanical Engineering and Laboratory of Computational Sensing and Robotics, Johns Hopkins University, 3400 N. Charles St. Baltimore, MD 21218. hopkinsaigroup@gmail.com

Fig. 1: We have collected about 1500 pictures of the robot from different angles and under different lighting conditions and labeled 380 pictures of them for training purpose.

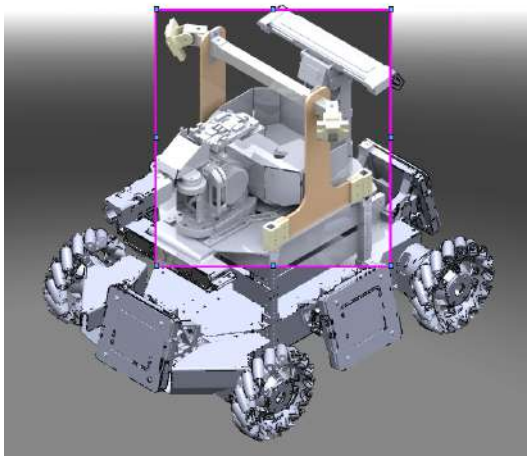
A. Mechanical Design

An ICRA 2018 DJI RoboMaster AI Robot, hereinafter called 'robot', is sponsored by RoboMaster Organizing Committee as a reward for approved technical proposal. This will be the only robot that we use for the challenge due to limited funding. Some custom designs and machining were made to accommodate the sensors and computer. Only two designs are addressed to conserve space for more important topics.

1) PrimeSense: A case and adapter is printed in white ABS to fix the camera in the front of the robot, as shown in the video. The adapter is attached at the bottom of the top platform that houses the firing mechanism and projectile feeding mechanism. The case was attached to the adapter by four bolt screws and nuts, clamping the camera tightly. This design provides an optimal field of view and great compactness while avoiding blocking the LiDar. Fig 2a shows the rendering of the design and how it integrates with the robot.



(a) Rendering of PrimeSense assembled on the robot



(b) Rendering of PrimeSense assembled on the robot

Fig. 2: Rendering Sensor Adapters

2) Camera: Beside the RGB camera in PrimeSense, two more cameras are implemented to enhance the perception ability of the robot. In order to achieve the best field of view as well as rigidity, an adjustable camera support was designed and manufactured, as shown in the video. The support sits on top of the top platform, utilizing existing poles as anchors. The laser-cut scaffold supports an extruded aluminum frame that the two cameras are attached to. 3D Printed adapters and casing allow two degrees of rotation up to 180 degree in each direction, while providing protection to the camera. Fig 2b shows the rendering of the design and how it integrates with the robot.

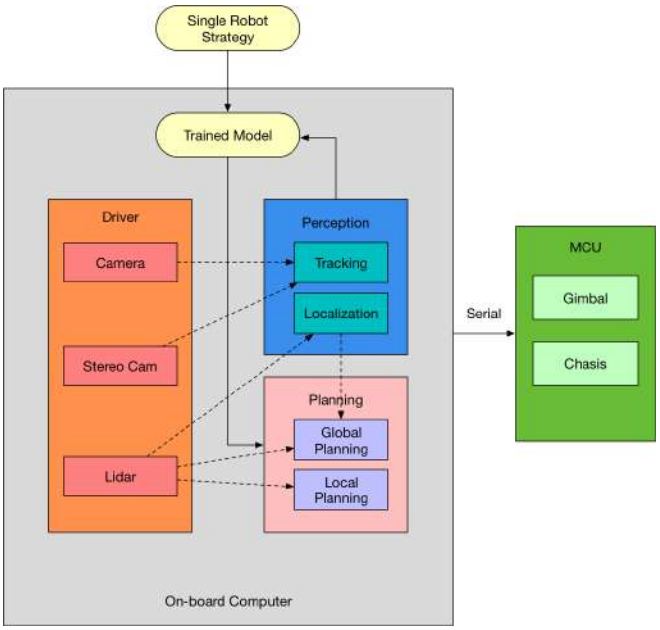


Fig. 3: System Architecture

(a) First Angle

(b) Second Angle

Fig. 4: Robot Navigating Mock Arena

B. Sensors

Sensor, as for robot is the eye for human. To make sure the robust performance of the Robomaster robot, we have gone through thorough research and found three different kinds of applicable sensors, which are, Stereo Camera, Lidar and Monocular camera, in consideration of the given configuration and limitation. For each of the sensors, we will elaborate the reason of choosing the current product as in the follow.

1) PrimeSense: Stereo camera offers Robomaster AI robot the ability to observe a scene in three dimensions. It translates these observations into a synchronized image stream (depth and color) just like humans do. Utilizing this sensor, the only concern was the bandwidth limitation from the only USB3.0 port of the Jetson TX2 onboard computer, considering the strategy of having

Field of View	57.5° 45° 69° (HVD)
Resolution and FPS	640×480@60fps

TABLE I: Important Parameters of PrimeSense

Field of View	80-120 degrees
Resolution and FPS	640×480@120fps
Type of Shutter	Electronic rolling shutter/Frame exposure

TABLE II: Important Parameters of the ELP USB Cameras

Range Radius	25 meters
Samples per Second	16000
Angular Field of View	50-360 degree

TABLE III: Important Parameters of the RPLIDAR

GPU	NVIDIA Pascal,256 CUDA cores
CPU	HMP Dual Denver 2/2 MB L2 + Quad ARM A57/2 MB L2
Mechanical	50 mm x 87 mm (400-Pin Compatible Board-to-Board Connector)

TABLE IV: Important Parameters of the NVIDIA Jetson TX2

multiple monocular cameras facing both sides of the robot. Thoughts were given, also, for realizing robot recognition and localization from collected 3D point clouds data of the robot, as different algorithms have proven the accuracy and precision of localization and object recognition. However, after implementing some algorithm to our own, we found out, to get a time-efficient result from a real time competition, would be considerably slow speed-wise, partly because of the objects are constantly moving, also given the complex structure of the robot with fairly amounts of surfaces to match with the cloud points data, mostly, we have not came to a fine optimization for the algorithm. To simplify, we decided to apply only the depth information into tracking and aiming enemy robot, for which, Kinect is of no necessity concerning the size. Finally, we have turned to PrimeSense, the product that has a smaller size, less bandwidth occupancy and, fair performance. Important technical specs are listed in Table I.

2) Camera: Two ELP USB cameras are installed on sides of the Robot to achieve a broad view. When selecting the camera, we mainly considered the following factors: resolution, frame rate, field of view, software compatibility, and price. In order to achieve better performance in detecting and tracking enemy, higher resolution and frame rate are usually desired. However, that doesn't mean we should go as high as possible. There are a couple of factors that we had to balance: Computation Power, Computer Bandwidth, Price. Though TX2 is powerful, its computation power is still limited due to its small size. What's more, there is only one USB 3.0 port that we could use to connect all the sensors, which posts a significant challenge for the bandwidth requirement. Finally, we only have limited funding for this project. Considering all the factors, two ELP USB Cameras were selected and integrated onto the robot. Important technical specs are listed in Table II.

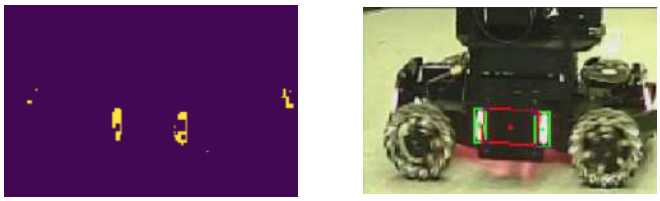
3) Lidar: In our design, the lidar is only used for localization algorithms. The small size of the arena leaves a lot of space for selection, however, higher precision is desired to achieve better performances. Based on the past experience of some of our team members, a RPLiDar is selected and integrated onto the robot. Important technical specs are listed in Table III.

C. Computer

When selecting the computer for the robot, we mainly took into account the size, the power, and the price. NVIDIA Jetson TX2 has a powerful configuration and small enough size, and was therefore selected as the main processing unit for the robot. Important technical specs are listed in Table IV.

D. Mock Arena

In order to test the hardware and the robot, we constructed a mock half arena that simulates a



(a) Extracted color areas. (b) Color detection results

Fig. 5: Color-based Armor Detection

half of the actual stage. The obstacles are made out of card boxes and duct tape acquired from HomeDepot. The fence was made out of poster boards from the 'robotorium'. Fig 4 shows the robot navigating the mock arena from two different angles.

III. SOFTWARE

A. Enemy Perception

This subsection talks about effort that we made in implementing different methods and algorithms to realize enemy detection. Utilizing limited resources, we attempted several strategies in order to accurately detect enemy to detect enemy pose and movement.

1) Color-based Armor Detection: To detect enemy armors, the most obvious way is to make use of the colored LED light panels. Apart from emitting bright red light, the LED panels possess some significant geometry attributes including width-height ratio, area and orientations, making them easily spotted in noisy environment.

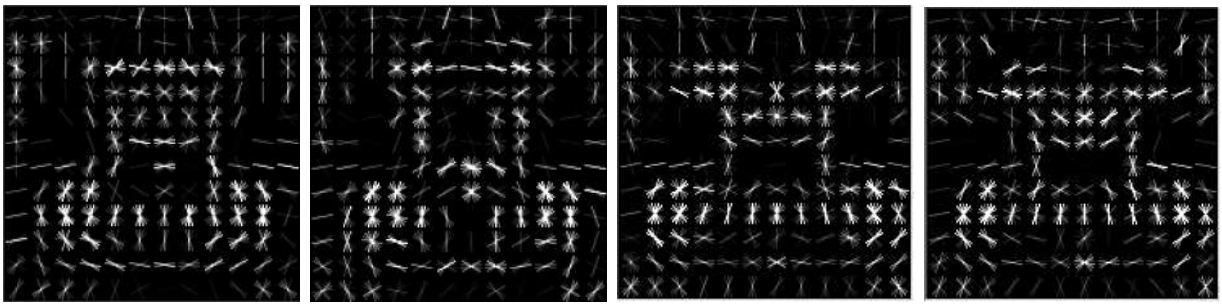
In order to extract the red colored LED lights from the image, we threshold each frame in HSV space, which is a more intuitive color space than RGB. In HSV space, the colored red LED has a hue value from 0 to 10 and 170 to 180. And because of the over exposure caused by the brightness of the lights, the intended area usually has a low saturation and a high value, sometimes even completely white near the center of the light panels, causing hollowed area in extracted color areas. These issues must be taken in to consideration when searching for a proper threshold. We then perform morphological opening on the resulting binary image from the color extraction, which would effectively eliminate the small noises while keeping the desired colored areas untouched. An example of the binary color mask is shown in Fig 5a.

With the colored areas extracted, their contours are easily detected and rectangles of a minimum size enclosing each colored area could be found based on the contours. The rectangles are then filtered based on the known geometry of the lights. Pairing candidates for lights with each other, we could find the resulting armor with the right attributes as shown in Fig 5b.

Calculating the pose of the armor is now a Perspective-n-Point problem with the armor geometry in Cartesian space known and the four corners of the armor calculated from the positions of the lights on the image. Mapping the 2D points into three-dimensional space, we could get an estimate of the position and orientation of the armor. Since this is a real time scenario, we choose the EPnP[1] algorithm from OpenCv's *solvePnP* method. The RANSAC variation of PnP algorithms is not applicable to this particular problem, since there is only four points. Something worth noting is that although the position from PnP is decent enough, the orientation calculated from this method is quite noisy and unstable because the color threshold is not always optimal to the lighting condition. Also, as the targeted enemy robot gets further away, it is increasingly difficult to tell the lights apart from environmental noise, especially the halo of the lights itself and the reflection of the LED lights from the floor. The current parameters we used for detection could achieve robust detection results within 4 meters, beyond that we need to rely on tracking methods which would be elaborated in later section.

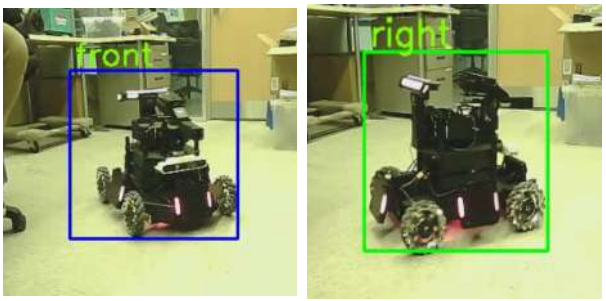
2) HOG + SVM Detection and KCF Tracking: The armor detection method described in the previous section has not achieve the level of performance we have in mind. So we decided that we need a vehicle detection and tracking module to help boosting the detection accuracy and robustness. Although it is tempting to use CNN based methods for the problem, we simply don't have enough man-power to collect and label the data we need, nor do we have access to the equally important computing-power for the training.

With limited resource, we turned to a less resource demanding method – Linear Support Vector



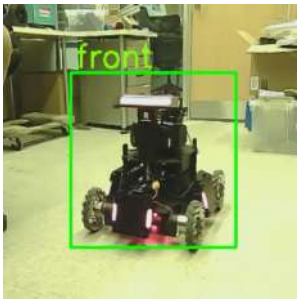
(a) Front HOG descriptor (b) Back HOG descriptor (c) Left HOG descriptor (d) Right HOG descriptor

Fig. 6: HOG descriptors



(a) Front correctly detected.

(b) Right correctly detected.



(c) Back wrongly detected as back.

(d) When detection fails, KCF tracking fills in.

(e) Vehicle occluded.

(f) Tracker recovered.

Fig. 7: Detection examples captured from a real-time detection test run.

Machines trained on the Histogram of Oriented Gradients (HOG) features as described in [2]. The method is used extensively in pedestrian detection and served as the de facto standard across many other visual perception tasks in the pre-CNN era.

The idea of HOG is to divide the image into multiple grids, and divide the gradient direction in each grid into several orientation bins. Gradients with greater magnitude contributes more significantly to its bin. Hence we have a histogram of gradient orientation of the grid, which can represent the structure of this grid. With all the grids together constitute a concise representation of the structure of the whole image while allowing some variance of the objects. Linear Support Vector Machines were then trained on the HOG features, which would find the hyper-plane that separates best the training examples and the negative examples. In our implementation, we all the area outside of the labeled bounding box are treated as negative examples. For the recognition part, the trained detector is applied on a pyramid of the input images, and then the final detection results are generated by performing non-maximum suppression on the bounding boxes from each layer of the pyramids.

One of the limitations of SVMs trained on HOG features is that it does not deal well with large rotational movements o f the object. To address this limitation, we trained four detectors for each side of the robot on merely 80-100 labeled images each. The trained HOG descriptors are shown in Fig 6. Unlike CNN-like methods, the descriptors appear to be quite interpretable. We can clearly see the shape of the wheels in all four descriptors, and the difference between the front view and the back view are significant, especially the gradients in the center where the turret is. The left and right

view are less differentiable, this reflects in the the results as that the detector can not tell the left side and right side apart. But this does not affect our goal, which is to detect the vehicle, not to find the orientation. To do this we apply the four detectors on each frame and choose the most significant one as output.

The result is surprisingly well with an 85% accuracy and a single instance of false positive on the test set. When run on real-time images captured from our USB camera, the detection result is satisfactory. Sometimes the detectors return the wrong side as shown in Fig 7c because the difference between each sides of the vehicle is not that different. Considering that training set is really small, this is acceptable for us.

To fill in the gaps between the detections where the detectors fail due to occlusion or large rotations, we apply Kernelized Correlation Filters (KCF) Tracking to the previously detected bounding box area. This is also the method we used in the previous section to tracking armors. The tracker performs well even on real time frames, together with our detectors, smooth detection and tracking is achieved. Moreover, as Fig 7e and Fig 7f suggests, the tracker is able to recover from occlusion.

We can apply the color-based detection described earlier in the output bounding box area only and this would speed up and enhance our detection.

3) Landmarks detection from Ensemble of Regression Trees: Beside the color-based armor detection, we are currently experimenting to align the detected vehicle, that is to say, to find its pose in another way. The method proposed in [3] is originally designed for face alignment, which is somewhat similar to our problem.

With the output from the method described in the previous section, we plan to apply this method to the detected region to find the landmarks points, and further solve the robot pose from the landmarks as a PnP problem. With significant features like wheels and lights, the landmarks of the robot from four sides of view is expected to be quite recognizable. We choose the four corner points of the visible armor, the two visible wheels from each side and the gun tip and the HP light bar as landmarks and trained and re-labeled the 380 labeled pictures we have. Some of the test results are shown in Fig 8.

Fig. 8: Landmarks (red dot) detected in test images.

This method is expected to have a more general result than the color-based method and to be robust against large illumination change. Since this needs to be trained separately based on the four detectors we have, the said method still needs more labeled data to deal with large rotations, especially for 45 degree views from each side.

B. Localization and Navigation

1) Localization: Localization is a version of on-line temporal state estimation, where a mobile robot seeks to estimate its position in a global coordinate frame.[4] Given adaptive particle filter, which converges much faster and is computationally much more efficient than a basic particle filter. To use adaptive particle filter for localization, we start with a map of our environment and we can make the robot start from no initial estimate of its position. Now as the robot moves forward, we generate new samples that predict the robots position after the motion command. Sensor readings are incorporated by re-weighting these samples and normalizing the weights. The package also requires a predefined map of the environment against which to compare observed sensor values.

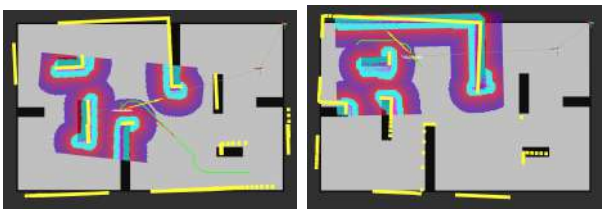


Fig. 9: AMCL localization simulation results.

At the implementation level, the AMCL package from ROS represents the probability distribution using a particle filter. The filter is adaptive because it dynamically adjusts the number of particles in the filter: when the robots pose is highly uncertain, the number of particles is increased; when the robots pose is well determined, the number of particles is decreased.

2) Navigation : Two key problems in mobile robotics are global position estimation and local position tracking. We define global position estimation as the ability to determine the robots position in an a prior or previously learned map, given no other information than that the robot is somewhere on the map. When navigating the robot through a mapped environment, the global trajectory is easily calculated by a graph search algorithm such as A*. Whereas the environment is not static and the trajectory needs to be constantly reiterated based on the current readings from the sensors. The local trajectory is optimized locally based on trajectory execution time, distance and heading difference wit respect to the goal, separation from the obstacles and the kinodynamic constraints of the robot itself in a manner called Timed Elastic Band method[5].

By knowing its global position, the robot can make use of the existing maps, which allows it to plan and navigate reliably in complex environments. Accurate local tracking on the other hand, is useful for efficient navigation and local tasks.

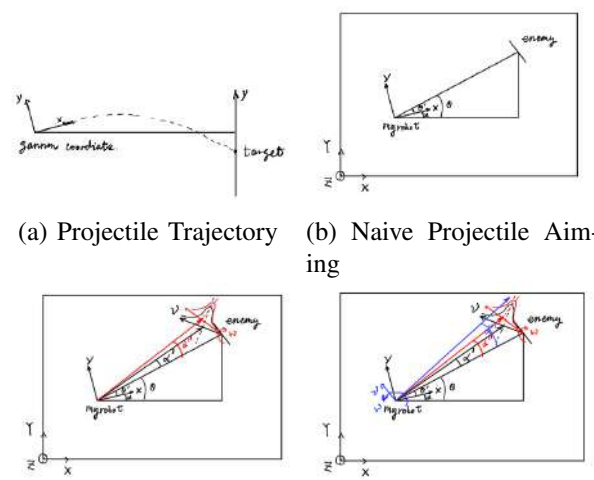
C. Aiming and Firing

1) Position and HP Check: First of all, we will aim at the enemy that is closest to our robot. When an enemy's HP is under a certain threshold, it will be prioritized to be killed.

2) The Physics Model of the Projectile: When projectile is flying in the air, its speed will decrease because of air resistance. So we need to build a function, whose input is the original speed and distance and whose output is the time it takes projectile to arrive at the target.

3) The Weight Setting of Distance and Area of Enemys Armor: The probability of hitting the target depends on the distance between our robot and the enemy, and the distance between two lights on the enemys armor. So we need to take both parts into consideration in order to decide which armor is the most valuable shooting target. If the target is close enough, it maybe very easy to hit. However, if the orientation of that target is quite tricky, the mission maybe fail. Therefore, we implement a weight function to decide the most valuable target.

4) Aiming at Enemys Armors: To start with, we assume that both target and our robot are static. Since we have already known the location of our robot and of enemy in the world coordinate, we can easily get θ, the relative angle from our robot to enemy in the world coordinate. Whats more, we have already known α, the position angle of our robot. So the pitch of the cannon θ' relative to frame of our robot is equal to $\theta - \alpha$. This circumstance is shown in Fig 10a. We also need to consider the pitch of the cannon, since the projectile will start falling before hitting the target. So we first calculate the distance between our robot and enemys armor. After that, we get the time the projectile needs to hit the armor. By using the



(a) Projectile Trajectory (b) Naive Projectile Aiming

(c) Adjusted Projectile Aiming (d) Optimized Projectile Aiming

Fig. 10: Projectiles firing strategies

formula $h = \frac{1}{2}gt^2$, the yaw can be solved.

5) Considering the Speed of Enemys Armor and Building a Gaussian Model (Without Rotation): When we consider the speed of enemy robot, we need to forecast the location of enemys armor. Denote the time when the projectile comes out of the cannon as T and the time it reaches the target as $T + T'$. T' can be calculated by distance between cannon and target divided by the speed of projectile. We assume that the speed of enemy conforms to Gaussian distribution and the speed is pure translation without rotation. So we get the offset α' (shown in Fig 10b), we add this offset to the pitch of the cannon.

6) Considering the Speed and Rotation of Enemys Armor and Building a Gaussian Model: Assume that the rotation of enemys armor also follows the Gaussian distribution. When we consider the rotation angle during the flying time of our projectiles, there will be a small offset α''. The final direction of pitch is show in the image as red lines in Fig 10c.

7) Considering the Rotation and the Speed of Our Robot: The speed and rotation of our robot only have effects on the speed of projectile. So in order to overcome these effects, we need a pitch offset. As shown in Fig 10d, the direction of the red line is the correct cannon direction, but its real direction is that of the blue line, which is caused by the speed and rotation of our robot. Therefore, we need to add the angle α''' between the two lines as the offset to our result.

8) The Physical Flying Model of the Projectile Extended Kalman Filter: Denote the position and orientation of enemy at time t as state X_t , where $X_t = [x_t \quad y_t \quad \theta_t]^T$. From CV part, the orientation and position of enemy with respect to our robot can be measured. Based on Extended Kalman Filter, we can accurately adjust our robot cannon to aim at enemy from measurement of enemy position and orientation.
Equations are shown as below

$$\bar{\mu}_t = g(\bar{\mu}_t, u_{t-1})$$

$$\bar{P}_t = G_t P_{t-1} G_t^T + R_t$$

Denote G_t as co-variance of State Gaussian Distribution at time t; μ_t as the mean of State Gaussian Distribution at time t; function $g()$ is the non-linear transition between states; u_t is the command signal at time t, where u_t is taken as spatial and angular velocity of Gimbals of our robot; R_t is the co-variance of measurement noise;

9) Dynamic Decision Making: Since the model is hypothetical, it should be adjusted in practice use. Some of the data is quite hard to obtain, such as the speed of enemy and the rotation of enemy. As result, we have to make reasonable hypothesis in which the observation during the competition and the attack strategy should be easy to change. To solve this problem, we also make other assumption, for instance, the speed of enemys robot is uniform distribution or Poisson distribution. Function models for different distribution are written just in case.

D. Strategy and Decision Making

This subsection talks about the effort and progress we made in achieving the autonomy of the robot. In recent years there have been much success of using deep representations in reinforcement learning. [6] Inspired by the work by OpenAI as well as the work done by Volodymyr Mnih in 2013[7] and 2015 [8], we decided to apply reinforcement learning, especially Deep Q-Learning(DQN) to teach the robot how to fight.

In order to apply any network, we would need an environment and a simulation which provides us with realistic enough rules and mechanism to mimic the actual challenge, while also offering sufficient control and fast training period by simplifying the real-world physics. To achieve this goal, we implemented two models.

1) Gazebo: The first simulation environment we attempted was Gazebo. Gazebo is a powerful tool. It perfectly simulates the actual challenge and also gives full control of the environment. In attempting to realize this, we built a simplified robot model as well as the arena. The figure below shows a screen-shot of the world and the robot in Gazebo.

Together with the stage and the robot, we also implemented a controller plugin for Mecanum Wheel mechanism and the projectile launching mechanism. However, as the team researched deeper into the project and OpenAI-Gym, a

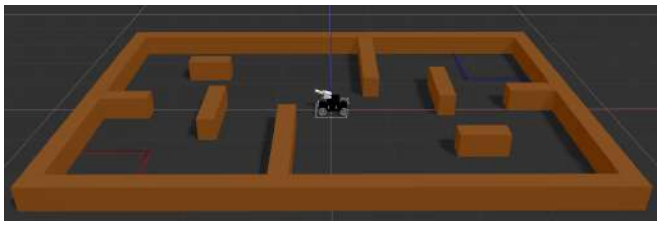


Fig. 11: Gazebo simulation of the stage and the robot.

reinforcement learning toolkit, we realized that the physics of the robot and the complicity of the challenge rules would make the learning and tuning period too long with respect to the time-line of this project and our limited access to computational power. Therefore, we pivoted towards a more simplified, more confined, and also more developed platform, Pygame. Though we did not use the Gazebo simulation to train our network from scratch, we still plan to use it to fine tune our AI module once we get a more matured model. At the same time, the simulation can also be used for testing perception algorithms, location algorithms, and navigation algorithms.

2) PyGame: After we decided to implement our learning algorithm in Pygame, we abstracted the game logic and constructed a discrete representation in the form of a Pygame program as shown in Fig 12.

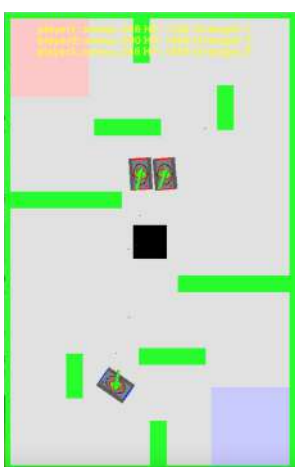


Fig. 12: A screen-shot of the GUI of the game developed in Pygame showing the basic elements of the game.

In the game, the player(us) controls a blue robot to fight two red robots that are reprogrammed to move around the center of the arena randomly. For training purposes, we discretized the arena into 50 x 80 different squares, with each indicates the same position in terms of game state. Each square represents an area of 10cm x 10cm square in real life. The purpose of applying DQN and implementing this game is to learn the positioning and moving strategy instead of aiming and firing. Therefore, the aiming and firing function is automated in the game. The cannon will automatically try to aim at the closest enemy with a Gaussian noise distribution with $\sigma = 2.5degree$. Meanwhile, a bullet will be fired at 18 m/s when the targeted enemy is in sight. The movement is discretized into 8 linear directions and 2 rotational directions, allowing both to happen at the same time. The velocity in each direction is calculated using either the mechanical maximum speed or the maximum speed indicated by DJI RoboMaster AI Challenge Rules v1.1. Each frame indicates a new state of the game, with a time-step equals to 0.1 second. Each robot is given 2000 hp and 300 ammo at the beginning of the round. Collision test will be run at each step to check if a robot will collide with obstacles or if a bullet hit a robot. When a bullet hits a robot no matter what position it hits, 50 damage will be dealt to that robot. A robot will be labeled dead and frozen when its hp drops to or below 0. The round ends when all robots on one side are labeled dead.

There are assumptions and limitations of the current game. First, even though the enemy movement is randomized, the enemy robots only move within a define area. This could result in specific strategy that only works for this scenario. However, this can be solved when we later feed the learned model into the AI robot behaviour. Second, the game did not consider acceleration. Therefore, it may result in a strategy which is physically impossible for the actual robot. We are currently monitoring the learned behavior and will decided if we need to add that feature. Third, the auto-firing feature assumes perfect perception algorithm. This could result in over-conservative model, however it can be mitigated by tuning the reward function and improving the perception algorithm.

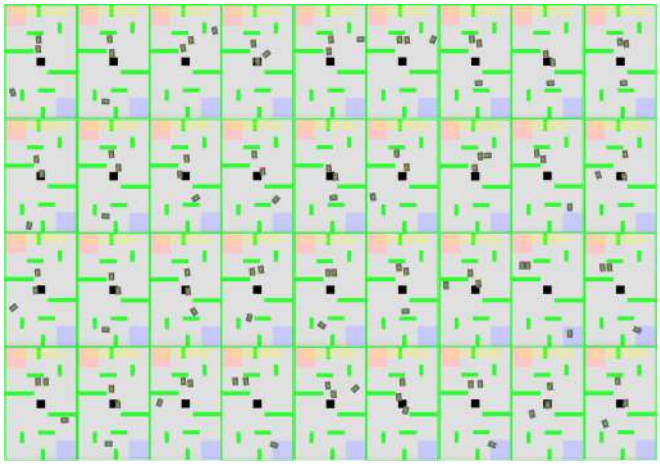


Fig. 13: Dueling DQN in training showing different states of the game.

3) Dual Deep Q-Learning Network: In this project, to avoid a huge Q table, we decide to use Deep Q Network for training AI strategy, which use Deep Network to replace Q table in reinforcement learning to select action. We create a memory database within our model to store the previous states, reward and action. By random select memory with 32 as batch size, we implement off-policy learning which have been proved to be promising. Also, the accelerate the convergence of network, we decide to use Dueling DQN[6], which decompose the Q into the value of state plus the advantage of each action. We use epsilon-greedy strategy as 0.9 without increment, decay rate as 0.9 and replace old target net with trained evaluation net every 200 iteration. Also we choose memory size as 500, and select observation state as self position and orientation, enemy position and orientation if in signt. We build the base deep-network as five fully connected layer, each with 500 nodes, and use RMSprop as optimizer.

Currently the training is still in progress. We are working to finetune parameters and reward function to optimize the performance. Fig 13 shows many screenshots of learning in progress.

IV. CONCLUSIONS

During the past months, the team members overcame many challenges and difficulties including heavy course and research workload, limited lab space, limited funding, and unavailable parts, etc. However, we are still able to achieve the presented results, plus many efforts that were not presented due to limited space. Throughout our R&D process, we introduced the RoboMaster challenge to the Hopkins community, and received widely positive feedback. Many students expressed their intention to join the team for next year's challenge, while some faculty members showed interest in utilizing this challenge as a research platform. Therefore, it is not only reasonable because of the progress we've made, but also important and beneficial for RoboMaster Competition to let us proceed to the final stage.

ACKNOWLEDGMENT

The project is supported by Prof. Charbel Rizk, Electrical and Computer Engineering department at Johns Hopkins University, and Prof. Louis Whitcomb, Laboratory of Computational Sensing and Robotics, Johns Hopkins University. We thank for their sponsorship and their lab space offering.

REFERENCES

[1] V. Lepetit, F. Moreno-Noguer, and P. Fua, "Epnp: An accurate o(n) solution to the pnp problem," *International Journal of Computer Vision*, vol. 81, no. 2, p. 155, Jul 2008. [Online]. Available: https://doi.org/10.1007/s11263-008-0152-6

[2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, vol. 1. IEEE, 2005, pp. 886–893.

[3] V. Kazemi and S. Josephine, "One millisecond face alignment with an ensemble of regression trees," in *27th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, United States, 23 June 2014 through 28 June 2014*. IEEE Computer Society, 2014, pp. 1867–1874.

[4] S. Thrun, D. Fox, W. Burgard, and F. Dellaert, "Robust monte carlo localization for mobile robots," *Artificial intelligence*, vol. 128, no. 1-2, pp. 99–141, 2001.

[5] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Trajectory modification considering dynamic constraints of autonomous robots," in *Robotics; Proceedings of ROBOTIK 2012; 7th German Conference on*. VDE, 2012, pp. 1–6.

[6] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*, 2015.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

[8] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529, 2015.